

SDP



Building Agent Systems

Subagents, Teams, and Autonomous Execution

Compose AI agents into systems that handle complex tasks, run autonomously, and improve over time

"What mechanisms make multi-agent AI work, and how do I compose them into autonomous systems?"

Single agents hit a ceiling: context fills, supervision is the bottleneck, quality degrades.

The solution is architecture, not a smarter single agent.

Developers

Subagent patterns, parallel execution, and autonomous hand-offs — no setup required

Architects

Design scalable agent systems using coordinator, worktree, and ensemble patterns



Engineering Managers

The business case: 9.7 hrs/week reclaimed per developer



300–500 LOC ceiling

Single agents hit a hard wall before most real features are complete

50–80% irrelevant context

One agent juggling research + planning + implementation by execution time

5–10× throughput gain

Background agents with worktrees vs. supervised single-agent execution

PART 1

Subagents

The Building Block

Context isolation, parallel execution, and the vocabulary to reason with

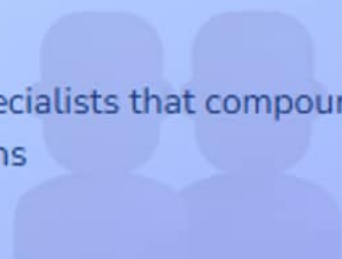


PART 2

Agent Teams

Organized Specialists

Squad: persistent role specialists that compound knowledge across sessions



PART 3

Autonomous Execution

Background Agents + Worktrees

Hand off to background agents — 85 active minutes become 27

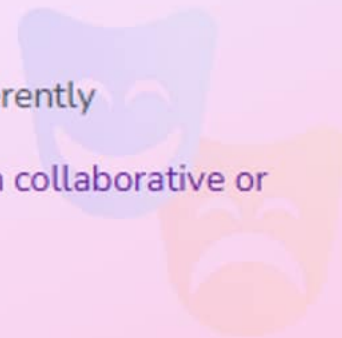


PART 4

Agent Council

When Models Think Differently

Claude + GPT + Gemini in collaborative or adversarial deliberation



Click any section to jump directly there

Subagents: The Building Block

The primitive mechanism that makes all multi-agent patterns possible



Context Isolation

Each subagent has its own 200K window



Parallel Execution

Independent tasks run simultaneously



Summary Returns

4-6x token reduction — findings only

Auth feature: research + analyze + implement
↳ 3 parallel subagents → 8 min vs 22 min serial

The Agent Decision Tree

Self-select your entry point — one branch lights per section of this talk

Simple task

Single-phase, under 200 LOC — plan mode handles it in-session

Plan Mode

Multi-phase

Research → analyze → implement with clean summary handoffs

Subagents

Role specialists

Planner + coder + reviewer with VS Code-enforced tool boundaries

Teams → S2

No supervision

Autonomous execution in its own branch — review when finished

Background → S3

Uncertain decision

Architecture choice with high stakes — stress-test across model families

Council → S4

Each branch maps to a section of this talk — the decision tree fully lights by the end

Two Ways to Invoke a Subagent

Implicit (Chat Hints)

Natural language delegation

Copilot infers intent and spawns the subagent automatically

Zero setup — describe delegation in chat

Best for ad-hoc research and exploratory tasks

Isolation guaranteed

All exploration stays isolated; only the summary returns

Explicit (Prompt Files)

Multi-phase .md files with frontmatter

tools, agents, and phases declared explicitly

Version-controlled — team-wide consistency

Best for reproducible and repeatable workflows

```
---  
name: feature-workflow  
tools: ['agent', 'read', 'edit']  
---
```

```
## Phase 1: Research  
## Phase 2: Implement
```

 Parallel mode: 3 independent subagents run simultaneously — 22 min serial becomes 8 min parallel.

Four Named Patterns



Subagents

Context isolation + parallel delegation — the primitive that powers every pattern that follows



Agent Teams

Role specialists via coordinator — planner, coder, reviewer each in their own context window



Background Agents

Autonomous execution in isolated git worktrees — supervision-free async work



Agent Council

Model diversity deliberation — Claude, GPT, and Gemini each bring different blind spots



Vocabulary before the tour — these four names give you the words to think with for the next 45 minutes.

Agent Teams: Organized Specialists

Role-based coordination via the coordinator-delegate pattern with Squad



Coordinator Pattern

Routes work, never implements directly



Tool Enforcement

VS Code validates — impossible, not just discouraged



Persistent Memory

Knowledge compounds across sessions

Team, build the authentication system

↳ 4 specialists in parallel – no context switching, no waiting

Roles, Tools, and Boundaries



Coordinator / Lead

Routes work to specialists, enforces reviewer protocol, selects models by cost — never implements directly

Scope + decisions

Reject → reassign (no self-review)



Dev Specialists

Frontend, backend, domain experts each in their own context with full edit and create access

Own 200K context

Parallel execution



Quality Roles

Tester + Reviewer with read-only and analysis tools — VS Code validates before invocation, impossible not just discouraged

Cannot modify files

Test + analyze only



10-minute setup with Squad — `npx github:bradygaster/squad` — persistent specialists living in your repo.

Knowledge That Compounds with Every Session

Four memory layers — agents start each session smarter than the last

charter.md	Identity, expertise, boundaries, and voice — read only by that agent	Per-agent
history.md	Project-specific learnings accumulated across real work sessions	Per-agent
decisions.md	Team-wide decisions all agents read before starting any task	All agents
skills/	Reusable patterns earned from real work — collective institutional IQ	All agents

Session 1: knows your stack. Session 5: knows your conventions. Session 20: knows your tech debt.

Squad Handles the Auth System

Fan-out to specialists — decisions captured in shared memory

```
$ copilot
> "Team, build the authentication system"
🧠 Copilot:
  📋 Ripley: Scoping requirements, defining API contracts ...
  🛠️ Dallas: Building login form and session UI ...
  🔧 Kane: Setting up JWT endpoints and token refresh ...
  📝 Lambert: Writing test cases from the spec ...

  📝 Lambert: Token refresh has a gap – edge case under concurrent load
  🔧 Kane: Picking it up – adding token nonce table

✅ decisions.md updated: JWT format, session storage, endpoint contracts
✅ 4 specialists ran in parallel – no context switching, no waiting
```

10 min setup → persistent team that learns your codebase

Autonomous Execution

Background agents + git worktrees — shift supervision from continuous to review-only



9.7 hrs/week

Active time reclaimed per developer



Git Worktrees

Each agent gets isolated branch +
filesystem



Zero Supervision

Plan → hand-off → review finished
work

10 tasks/week with background agents
↳ 270 min active vs 850 min supervised

9.7 Hours Reclaimed Per Week

10 tasks $\times$ 58 minutes saved per task — the EM math that turns a demo into a budget conversation

9.7 hrs

reclaimed per developer per week with
background agents

10 tasks $\times$ 58 min saved: 850 min supervised $\rightarrow$ 270 min active

 **85 $\rightarrow$ 27 min per task**

Plan (15) + hand-off (2) + review (10) replaces plan + supervise + review

 **3 parallel tasks: 51 min total**

vs 255 min serial supervised — 5 $\times$ faster for independent work

 **Built into VS Code 1.109+**

Session picker + worktree creation built in — zero additional tooling

 **Auth runs unsupervised**

Hand off the auth system — review the finished branch when done



Business case first: 68% of your developers' implementation time returned to them — not a demo convenience.

Supervised vs Background Agent

Same output — 68% less of your time

✗ Supervised Execution

- 1 Plan the task
15 min to define scope and approach
- 2 Actively supervise
60 min answering questions, monitoring progress
- 3 Answer mid-task questions
Cannot parallelize — supervision is the bottleneck
- 4 Review output
10 min — often re-explaining intent

 85 min active — continuous supervision required

✓ Background Agent

- 1 Plan the task
15 min — identical, same quality
- 2 2-min hand-off
Explicit criteria + constraints → agent runs in worktree
- 3 Do other work
Agent executes autonomously — you are unblocked
- 4 Review finished branch
10 min — criteria were explicit, output matches intent

 27 min active — 68% reduction in developer time

10 tasks/week: 850 min supervised → 270 min with background agents

Each Agent Gets Its Own Branch

Shared Workspace

File conflicts guaranteed

Multiple agents editing the same files simultaneously

90 min to revert a failed experiment from shared history

Agent B can see Agent A in-progress — true isolation lost

One agent at a time or accept collision risk

Git Worktrees

Zero conflicts by design

Agent A and Agent B both edit main.js — different worktrees

Failed experiment removed in 5 min — just delete the worktree

VS Code 1.109 creates worktrees automatically on agent start

Review branch → merge if successful, remove if not

0

conflicts across parallel agents

5 min

to discard a failed experiment

3×

agents editing same file safely

Three Parallel Execution Patterns



Independent Features

Feature A, B, C → worktree-a, -b, -c running simultaneously

Plan all 3 upfront

Review 3 finished PRs



Experimental Variants

Redis vs LRU vs hybrid — 3 real implementations, benchmark decides the winner

Empirical, not speculative

Discard losers instantly



Parallel Roles

Implement + write migration tests + update docs simultaneously in 3 worktrees

Merge in dependency order

No sequential waiting



CLI fan-out: `/fleet implement Feature A in auth/, Feature B in payments/` — orchestrates all patterns from the terminal.

PART 4

Agent Council

When Different Models Think Differently



Collaborative Mode

Agents enrich each other's drafts



Adversarial Mode

Models debate to stress-test decisions



Auth Flaw Caught

Council surfaces what 3 layers missed

```
stress-test: Review the auth implementation for production readiness
↳ Flaw caught. Verdict: OVERTURNED.
```

Two Modes, One Install

👉 Collaborative 👉

Phase 1 (parallel)

Alpha, Beta, Gamma each draft independently

Phase 2 (parallel)

Each reads the other two drafts and improves

Phase 3

Orchestrator synthesizes all three enriched perspectives

7 model calls total

Wall-clock $\approx$ 2 sequential subagent calls

✂️ Adversarial ✂️

Phase 1 (parallel)

Alpha, Beta, Gamma tackle the question independently

Phase 2

Orchestrator identifies the strongest position

Phase 3 (parallel)

Other two agents attempt to tear it apart

Phase 4 — Verdict

SURVIVED / MODIFIED / OVERTURNED + rationale

🔑 Trigger auto-detected: council:/brainstorm: → collaborative; debate:/stress-test:/vs → adversarial.

The Flaw Surfaces Under Deliberation

Threaded through S1 → S2 → S3 — council finds what every prior layer missed

```
$ copilot
```

```
> "stress-test: Review the auth implementation for production readiness"
```

🤖 Copilot:

Alpha: JWT refresh has a race condition under concurrent load – simultaneous requests generate multiple valid tokens simultaneously

Beta: Confirmed race condition. Token nonce is 8 bytes – entropy insufficient, collision guaranteed at 500+ concurrent users

Gamma: Session invalidation also affected – logout cannot guarantee all tokens revoked across distributed instances

✅ OVERTURNED: Two critical auth bypass vectors identified under load

✅ Action: distributed nonce table + advisory lock on refresh endpoint

The human approved the Squad artifact. They didn't attend the meeting.

Deliberation → decisions.md → Execution



AgentCouncil Deliberates

Stress-test architecture decisions before committing — model diversity surfaces blind spots no single model catches

Adversarial for high-stakes decisions

Collaborative for design exploration



decisions.md Records

Council verdict written to shared memory — all Squad agents read it before starting the next task

Architecture governs itself

Human approved the artifact



Squad Executes

Coordinator routes the validated decision to role specialists — implementation starts with the decision already made

No redundant deliberation

Persistent memory compounds



The feedback arrow: council output feeds autonomous execution. Deliberation → Execution is the complete loop.

From Single Agent to Multi-Agent Architecture

Before

Single agent juggling research, planning, and implementation

50–80% of context is irrelevant by execution time

Continuous supervision required — cannot look away

Multiple agents sharing workspace create file conflicts

After

Coordinator delegates to role-specialized subagents in parallel

Only 100–500 token structured summaries return to main context

Background agents execute autonomously in isolated worktrees

AgentCouncil stress-tests decisions before execution commits

9.7 hrs/wk

active time reclaimed with background agents

5–10×

throughput with parallel specialized execution

0 conflicts

with git worktree isolation per agent

✓ What You Can Do Today

Today

- Try implicit subagent: Run a subagent to research X, return a structured summary
- Run 3 parallel subagents on a PR: security, performance, and testing in parallel
- Hand off one task to a background agent via VS Code session picker and review the branch

This Week

- Install Squad with `npx github:bradygaster/squad` and form your first team
- Write `charter.md` files for each role: Lead, Dev, Tester
- Install AgentCouncil and run one adversarial debate on a real architecture decision

This Month

- Move 10 tasks/week to background agents and measure the active-time reduction
- Establish `decisions.md` as persistent team memory that compounds across sessions
- Use council deliberation before any architecture decision you will live with for years

🔑 Key Takeaway

Compose agents the way you compose a team: the right specialist, in the right isolation, at the right time.

 VS Code Documentation[Subagent Invocation and Parallel Execution](#)

Context isolation, parallel subagent patterns

[Custom Agents in VS Code](#)

YAML frontmatter, tool scoping, model selection

[VS Code v1.109 Agent Orchestration](#)

Invocation controls and parallel agent support

[Background Agents Documentation](#)

Hand-off workflow and worktree integration

[VS Code v1.109 Agent Session Management](#)

Session picker and multi-agent monitoring

[Git Worktrees Reference](#)

Technical reference for worktree lifecycle

 Production Implementations[Squad — Production Agent Team System](#)

Persistent memory, parallel execution, GitHub Issues integration

[AgentCouncil — Multi-Model Deliberation](#)

Claude + GPT + Gemini collaborative and adversarial modes

[Agentic SDLC](#)

Repo, CI/CD, and org-wide patterns for agents at scale

[Copilot Primitives](#)

Instructions, prompts, skills, and agents — the building blocks



Building Agent Systems

Subagents, Teams, and Autonomous Execution



9.7 hrs/wk

Active Time Reclaimed

10 tasks × 58 min saved with background agents



Auth Flaw Caught

Council Surfaces What Layers Miss
Model diversity as a quality mechanism



decisions.md

Architecture Governs Itself
Human approved the artifact — didn't attend the meeting

What's your first task to hand off to a background agent this week?